

In Figure 6, Lines 129 to 133 make it clear that the gcc compiler is generating the code which wraps around and goes to the negative side.

Let us compile program-4 using the gcc compiler with the options `-fstrict-overflow` and `-fno-strict-overflow`, and see how the compiler generates the assembly codes.

 Note 1: How to compile the code using options and generate the assembly code

1. To compile the code with `-fno-strict-overflow`:
`gcc -g -fno-strict-overflow program_4.c`
2. To compile the code with `-fstrict-overflow`:
`gcc -g -fstrict-overflow program_4.c`
3. After compiling, to save the assembly code separately:
`objdump -S a.out > 1_no_strict_overflow`
`objdump -S a.out > 1_strict_overflow`
4. Comparing both the files using the `vimdiff` command:
`vimdiff 1_no_strict_overflow 1_strict_overflow`

The output will be what's shown in Figure 7.

 **Note 2:** The `-fstrict-overflow` option allows the compiler to assume strict signed overflow rules, depending on the language being compiled. For C, this means that when doing arithmetic with signed numbers, the overflow is undefined by the C standards, so the compiler may assume that it does not happen. This permits various optimisations. For example, the compiler may assume that an expression like `'x + 2 > x'` is always true for signed `'x'`. This assumption is only valid if signed overflow is undefined, as the expression is false if `'x + 2'` overflows when using two's complement arithmetic. When this option is in effect, any attempt to determine whether an operation on signed numbers overflows must be written carefully to not actually involve overflow.

The `-fno-strict-overflow` option allows the compiler not to assume strict signed overflow rules.

In Figure 7, we can see the difference between the assembly code when compiled with the options `-fno-strict-overflow` and `-fstrict-overflow`. The left side assembly shows that code is generated using the option `-fno-strict-overflow`, in which the gcc compiler will ignore the signed integer overflow conditions and resume with normal code generation. But, on the other hand, the right side assembly code shows the code is generated using the option `-fstrict-overflow`, in which the compiler will apply all the overflow rules.

Instance 2: Similar to the example shown in Instance 1, one more instance can be shown where adding two numbers results in the overflow condition, which leads to undefined behaviour.

```
1 #include <stdio.h>
2
3 int main()
```

```
4 {
5     /* Declaring two int variables */
6     int a;
7     int b;
8
9     /* Code to read the values of a and b */
10
11     /* Calling add function */
12     int sum = add(a, b);
13
14     /* Some code here */
15
16     return 0;
17 }
18
19 /* Function definition */
20 int add(int a, int b)
21 {
22     int sum;
23
24     sum = a + b;
25
26     return sum;
27 }
```

In the code shown in program-5 above, in Line 24, the result may overflow after adding two numbers, since both the variables are signed by default, which leads to undefined behaviour.

Modifying the value of the variable more than once between two sequence points ends up in undefined behaviour. Modifying the string literal, which is pointed to by the pointer, also leads to undefined behaviour since the standard imposes absolutely no requirements and the compiler can do anything. According to the C standards, signed integer overflow is undefined behaviour too. A few compilers may trap the overflow condition when compiled with some trap handling options, while a few compilers simply ignore the overflow conditions (assuming that the overflow will never happen) and generate the code accordingly. **END** 

References

- [1] <http://c-faq.com/ansi/undef.html>
- [2] http://clang.llvm.org/get_started.htm
- [3] <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1256.pdf>

By Satyanarayana Sampangi

The author is a member of the embedded software team at Emertxe Information Technology (P) Ltd (<http://www.emertxe.com>). His areas of interest are embedded C programming combined with data structures and microcontrollers. He can be reached at satya@emertxe.com.